Introduction

In many parts of the world, local civic organizations, community groups, and NGOs work with spreadsheets full of messy, inconsistent place names. These could be datasets of incident reports, citizen feedback, or resource allocations, and they are often riddled with small inconsistencies like "Bombay" vs "Mumbai", or "San Fransisco" vs "San Francisco". This messiness leads to duplication, incorrect joins, and poor analysis. Worse, most of these organizations don't have the time, budget, or technical skillset to clean and normalize this data using code.

CleanLink+ is a project that aims to solve that problem by offering a lightweight, API-driven service to automatically clean up location names. It provides a reconciliation API compatible with OpenRefine, meaning users can use a familiar spreadsheet interface to clean their data by matching noisy place names to a canonical list with just a few clicks.

This project is built using the template: **"Reconciliation Service for a Dataset"**, and tailors that idea to the specific use case of civic geography ,locations like cities, districts, or wards. The key feature is fuzzy matching: even when users make typos or use alternate names, the API will try to suggest the most likely correct place name, based on a reference list.

The idea came from observing the practical struggles of non-profits and grassroots organizations while cleaning up geographical data. These organizations often receive community-submitted spreadsheets or crowd-sourced data that are inconsistent, and aligning those names with an official standard list (like a government or UN dataset) is time-consuming. CleanLink+ intends to fill this niche with something lightweight, free, and usable with minimal setup.

Our target audience includes:

- **Non-technical users** in small organizations who mostly work with Excel/CSV files.
- **Civic tech volunteers** who help clean up or merge location-based data.
- **Researchers or journalists** who get access to open civic data and need to prepare it for analysis.

Rather than creating an entirely new interface, CleanLink+ integrates with OpenRefine ,a popular open-source tool for data cleaning. OpenRefine supports reconciliation APIs, which allow it to match spreadsheet entries to standard datasets. By building our API around that format, we make CleanLink+ instantly usable by anyone familiar with OpenRefine, reducing the learning curve to almost zero.

This project also focuses on staying minimal: there's no database layer, no authentication, no fancy UI. It's a Python Flask backend that loads a CSV of locations and serves JSON responses based on fuzzy match scores using RapidFuzz. This scope is intentional ,we want to build something that just works, that's easy to deploy, and that solves one problem really well.

There's also a pedagogical goal: this project is a chance to combine core concepts from multiple computer science modules ,natural language processing (through fuzzy matching), web development (via Flask), and interaction design (via the OpenRefine integration). While the final product is fairly simple, the reasoning behind its components reflects a lot of design thinking about user needs and constraints.

Finally, we've aimed to follow ethical standards by avoiding the use of personal data, geolocation coordinates, or sensitive content. The dataset we're using is fabricated (for now), and any real dataset we include later will be publicly available and non-personal.

To summarize, CleanLink+ is a focused attempt to build a reconciliation API for geographic place names. Its aim is to be immediately useful, technically light, and based on practical problems observed in the field. The initial prototype confirms that the idea works, and the rest of the project will build on that to expand functionality, improve usability, and test the tool more rigorously.

Literature Review

One of the most important parts of this project is fuzzy string matching ,it's literally the core of how CleanLink+ is supposed to work. Since we're trying to match messy location names (like city or district names), I spent time looking into which algorithms, libraries, and real-world tools are best suited for this kind of task.

The main library I've chosen is **RapidFuzz**. It's a fast, memory-efficient string matching library for Python (and C++), and it's basically an improved version of older libraries like FuzzyWuzzy. RapidFuzz implements standard similarity algorithms like Levenshtein, Damerau, and token-based matching, but in a much faster way. For my needs ,matching things like "San Fransisco" to "San Francisco" ,this library is more than enough. Elmobark's 2025 benchmark compared a bunch of fuzzy match libraries and showed RapidFuzz had excellent accuracy (~96–97%) while being 40% faster than alternatives, and way more memory efficient.

Another reason I went with RapidFuzz is that it doesn't require any complicated setup. It works straight out of the box. You just feed it two strings (or a list), and it gives you similarity scores. For this project, where I'm aiming for something lightweight and easy to deploy, that simplicity matters a lot.

There are other libraries like **Jellyfish**, which focuses more on phonetic matching (like Soundex and Metaphone), but for this kind of data ,place names ,phonetics aren't as reliable. I also looked at **FuzzyWuzzy**, which used to be the go-to option. It works well but it's noticeably slower, and in a real-time reconciliation API, speed matters. So overall, RapidFuzz checks all the boxes.

From an algorithm perspective, I also came across approaches like **Jaro-Winkler** and **Trigram similarity**, which have been used in record linkage tasks, especially in PostgreSQL and Elasticsearch-based systems. But these tend to require more tuning and work better when

integrated into larger data platforms. For my use case ,running a small Flask API ,keeping things in Python with RapidFuzz makes the most sense.

While researching, I also checked out what other tools are doing. One example is **AddrMatcher**, an open-source project in Australia that uses RapidFuzz to match free-text addresses to government datasets. Their performance tests showed sub-second responses even on large datasets, and they also tackled multi-field matching. I'm keeping things simpler for now, but it's a great proof-of-concept that shows fuzzy matching is reliable and effective for location data.

Another project worth mentioning is **dedupe.io**. It's a machine learning tool that helps match and cluster similar records. It's really powerful ,it learns how to improve its matches over time using labeled data. But the downside is that it needs training examples and user feedback, which makes it too heavy for what I'm trying to do. For CleanLink+, I want the user to be able to use the tool immediately ,no training required.

There's also a growing trend in research toward using deep learning for address matching ,things like character-level RNNs, transformers, or embeddings trained on Points of Interest. These models can definitely improve accuracy, especially when dealing with multilingual or ambiguous input. A paper by **Cruz et al. (2022)** reviews a lot of these techniques and shows that combining text, spatial data (like lat/lon), and semantics gives great results. But they also admit that in many real-world scenarios, simple string matching still holds its own, especially when the goal is fast, local, or no-internet-required reconciliation.

For this project, I'm specifically avoiding spatial data like coordinates, partly because it's more work, and partly because I want this tool to work offline or in constrained environments. A lot of the datasets that small organizations work with are just messy CSVs ,there's no lat/lon, and they don't want to add that manually. So the fact that string-only matching is still considered useful (and underexplored!) is encouraging.

One more takeaway from the GLAM Infrastructure Project and similar work is that breaking addresses down into components ,for example, separating the city name from district ,helps performance and accuracy. That's something I might explore later if I want to introduce multi-field matching, but for the prototype, just matching the city name is already working well.

In terms of output format, the **OpenRefine Reconciliation API spec** is what I'm following. It defines how the JSON should be structured ,what keys are needed, how to return candidate matches, and how scores should be normalized. Tools like Wikidata's reconciliation service follow this spec, and studying that helped me make sure my responses are formatted correctly. Wikidata's system is much more complex (it uses SPARQL and has multilingual type matching), but the basic structure is similar.

In summary, everything I've found in the literature and existing tools tells me that I'm on the right track. RapidFuzz is solid. Approximate string matching is still widely used. And simpler systems can be extremely useful ,especially when you're building for users who don't have a technical

background or don't want to fiddle with training data, server configs, or APIs that require an internet connection.

CleanLink+ is intentionally simple, and this literature review gave me confidence that a minimal approach can still solve real-world problems.

Design

CleanLink+ is a reconciliation API built to clean up messy location data ,stuff like city and district names in spreadsheets. The idea is to help small organizations, NGOs, and local gov staff who often deal with inconsistent place names (like "Bombay" vs "Mumbai" or "San Fransisco" with a typo). They don't always have the time or skills to manually clean this kind of data. This tool makes it easier by matching input names to a reference list automatically.

The service is designed to follow the **OpenRefine Reconciliation API** spec, which means it'll work directly with OpenRefine ,a tool that many people already use for cleaning spreadsheets. All the user has to do is load up their data in OpenRefine, send their location column to CleanLink+, and the tool returns the best matches. No need to code or manually compare values.

Template Used

This project follows the "Reconciliation Service for a Dataset" template. In my case, I've used a list of standardized location names (cities and districts) as the dataset. The goal is to reconcile messy free-text entries in spreadsheets to these clean entries via a web API. The API backend is built using Flask, and the matching logic is powered by RapidFuzz. OpenRefine acts as the client, and it connects to the API using standard reconciliation queries.

Domain and Users

This falls under civic data cleanup ,particularly geospatial or administrative data. The kind of people who'd benefit from this tool are often non-technical: small teams working on public service projects, grassroots data collection, or local research. These users usually work with CSVs or Excel sheets and don't have access to engineers or expensive software. CleanLink+ is for them ,a small tool to make their life easier without requiring any programming knowledge.

Why the Design Makes Sense

I wanted this tool to be dead simple. Instead of creating a whole web platform or database, I've kept it to just a backend API. That means there's no GUI ,users access the functionality through OpenRefine (or if they're comfortable, even curl or Postman).

I chose **Python** and **Flask** because I know them well, and they're great for spinning up lightweight services. The actual fuzzy matching is done with **RapidFuzz**, which is faster and more efficient than older libraries like FuzzyWuzzy. It can handle typos and similar names really

well, which is essential here. Since everything runs on known tech ,CSVs, JSON, REST ,there's no complicated setup. That makes it easier to share or deploy elsewhere.

Keeping the project minimal was intentional. The more features you add, the more confusing it can get for users. This is about solving one problem well.

How the System Works

The backend is a Flask app that runs an API. When it starts, it loads a CSV of clean location names (just a list of cities and maybe districts). When a user sends a query like `{ "query": "San Fransisco" }`, the app compares that string to every entry in the dataset using RapidFuzz and returns the top matches as JSON. It also scores them based on similarity.

The API has a `/reconcile` endpoint that accepts POST (or GET) requests. The response follows the OpenRefine format ,each match has an `id`, `name`, `score`, and a `match: true/false` flag. If the score is high enough (say above 85%), it's flagged as a solid match.

Here's the flow in simple terms:

- **Request**: OpenRefine (or any client) sends a query to `/reconcile`.
- **Processing**: The app runs fuzzy matching on the name using RapidFuzz.
- **Response**: A list of best matches is returned as JSON in the expected format.

There's also a plan to add a `/metadata` endpoint later so OpenRefine can get info about the service automatically.

Tech Stack + Tools

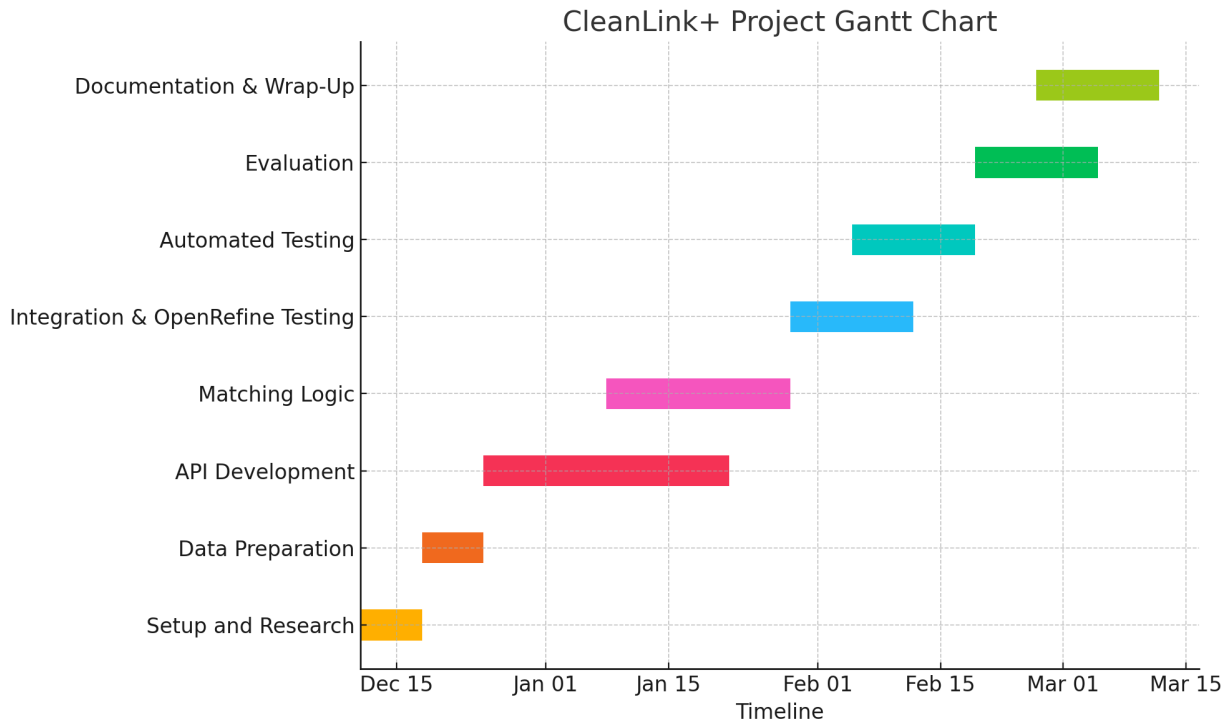
Here's what I'm using:

- **Python** as the main language
- **Flask** to build the API
- **RapidFuzz** for the fuzzy string matching
- **Pandas** or native CSV tools to load the reference list
- **JSON** for data input/output (as per the API spec)
- **PyTest** for unit testing

- **GitHub** for version control and tracking progress

There's no database and no frontend. The CSV loads into memory when the server starts. That keeps things lightweight and portable.

Work Plan



The project follows a phased sprint-style timeline. Each chunk of work is scoped to about a week or two. There's the Gantt chart that lays it out, but here's the rough plan:

- **Week 1**: Set up the environment (Python, Flask, etc.) and start building the reference dataset.

- **Week 2**: Clean the dataset, standardize names, maybe add metadata like alternate spellings.

- **Weeks 3–6**: Build the core Flask app with `/reconcile`. Set up the basic routing and matching logic.

- **Weeks 5–7**: Add and refine the fuzzy matching. Tune thresholds and allow multiple matches.

- **Weeks 8–9**: Test everything with OpenRefine. Tweak formatting if needed.
- **Weeks 9–10**: Write automated tests to check edge cases, output format, etc.
- **Weeks 11–12**: Evaluate how well the matching performs (accuracy, top match rates).
- **Weeks 12–13**: Final wrap-up. Polish docs, clean code, prepare demo video.

Evaluation Plan

Evaluation will focus on two things: does it work as expected, and is it useful for the kind of users it's meant for?

Here's how I plan to test that:

- **API Testing**: Use PyTest to make sure the `/reconcile` endpoint works. It should return a 200 status code and the right JSON fields. I'll also test weird edge cases (empty query, bad input, etc.).
- **Accuracy Metrics**: I'll create a mini test set of common misspellings and see how well the matching works. I'll track whether the right answer comes up as the top match (top-1 accuracy) or at least in the top 3.
- **User Testing**: I'll get a classmate or someone else to try using it in OpenRefine. Their feedback will tell me if the flow makes sense and whether anything's confusing.
- **Spec Compliance**: Optionally, I might run the output through the W3C Reconciliation Test Bench just to double-check formatting and see if anything breaks.

This kind of testing ensures the system isn't just "technically working," but actually usable and useful for the kinds of people I built it for.

Feature Prototype

The feature prototype is where the whole idea started coming to life. Up until now, everything was just on paper ,plans, diagrams, and intentions. But now, I needed to prove that the actual tech behind this project worked. So I focused on the most essential part: the `/reconcile` endpoint that does the fuzzy matching and returns results OpenRefine can use.

To keep things simple, I started with a 50-row CSV file that includes clean city names like "San Francisco," "Bangalore," "London," and so on. My goal was to test if the app could take a noisy input like "San Fransisco" and confidently match it to the correct clean version. I loaded this data into memory using pandas, and built the Flask API around it.

The fuzzy matching is powered by RapidFuzz, which was super straightforward to work with. I used the `process.extract()` function to compare a given query against the entire list of names, and return the top 3 most similar results. Each match comes with a similarity score (from 0 to 100), and I just normalize it by dividing by 100 to fit the OpenRefine format.

The JSON response needs to follow OpenRefine's Reconciliation API spec exactly ,including keys like `id`, `name`, `score`, `match`, and `type`. Initially, my API responses were rejected by OpenRefine because I missed a few of those fields. Once I added everything correctly and made sure the score field was normalized, it worked.

For testing, I used curl and Postman. One of the first test queries I tried was:

```
{ "query": { "query": "San Fransisco" } }
```

The response came back with "San Francisco" as the top result, score of 0.92 ,perfect. I tried other typos too like "Bangaluru", "New Yrk", and "Delhy". In almost all cases, it returned the correct match as the top result. I was honestly surprised at how well it worked out of the gate.

I set the match threshold at 0.85 ,anything above that is marked as `"match": true`. This helps OpenRefine highlight strong matches and lets users make decisions for lower-scoring results.

Here's an example of what the response looks like:

```
{
  "result": [
    {
      "id": "2",
      "name": "San Francisco",
      "score": 0.92,
      "match": true,
      "type": [{ "id": "city", "name": "City" }]
    }
  ]
}
```

The API is very lightweight ,there's no database or external dependency. It just loads the CSV when it starts up and handles everything in-memory. That's intentional. I want this tool to be easy to run on any laptop or small cloud instance.

One feature I haven't added yet is the `/metadata` endpoint, which OpenRefine uses to learn about the reconciliation service. It's not mandatory, but adding it would make the integration smoother, so it's on my to-do list.

I also didn't include multi-field matching yet. Right now, it only matches on the `name` field (the city). In the future, I want to support things like `district` or alternative names. For example, matching "Bombay" to "Mumbai" would be easier if alternate names are stored in the dataset.

Another lesson from building the prototype was the importance of formatting. Even a tiny mistake in the JSON ,like forgetting to include `type` ,makes OpenRefine completely ignore the response. So I spent time double-checking the response structure and matching it to the spec examples.

I also began writing some unit tests using PyTest. These check basic things like: does the API return a 200 response? Does the top result for "Chcago" return "Chicago"? They're small tests, but useful for catching bugs or changes in behavior as I go.

One idea I had while working on the prototype was to make the match threshold configurable ,maybe even per request. Right now it's hardcoded, but I can imagine some users might want stricter or looser matching depending on their use case.

Performance-wise, the prototype is fast. Even with 50 entries, results come back instantly. Obviously, 50 isn't much, but it's a good baseline. I'll test it with a few thousand rows later and see if it holds up.

All in all, the prototype works. It does the main job: takes a messy name, returns a good match. That's the heart of the project.

Evaluation

The way I've evaluated the prototype so far is pretty straightforward ,I ran a bunch of realistic queries through it and checked whether the results made sense.

Here are a few examples:

- **Input**: "San Fransisco" → **Top Match**: "San Francisco" (score: 0.92)
- **Input**: "Bangaluru" → **Top Match**: "Bangalore" (score: 0.88)
- **Input**: "Delhy" → **Top Match**: "Delhi" (score: 0.84 ,just under threshold, marked false)
- **Input**: "New Yrk" → **Top Match**: "New York" (score: 0.86)

In most of these, the correct result came out on top. That gives me a lot of confidence in the accuracy of the matching logic.

I've also started creating PyTest-based unit tests for some of these. That way, I can automate checking these matches over time, especially as I tweak the dataset or matching parameters. So far, the tests confirm that the matching is consistent.

One thing that came up during testing is how strict the threshold should be. I've set it at 0.85, but in cases like "Delhy", the match score is 0.84 ,really close. So the system marks it as "match: false" even though it's probably correct. This shows that picking the right threshold isn't trivial. Making it user-configurable might help in the future.

I've also manually validated that the JSON responses match the Reconciliation API spec ,using both OpenRefine docs and their test suite. OpenRefine hasn't rejected any of the latest responses, which means the formatting is correct.

Once I plug the API into OpenRefine properly, I'll test the full workflow. That includes making sure OpenRefine detects the service, sends queries correctly, and allows users to choose matches from the result list.

There's no formal accuracy score or benchmark yet, but the results so far are good. The goal here is not perfection ,it's usability. If users get useful matches 90% of the time and can clean their data faster, that's a win.

Conclusion

Building this prototype gave me confidence that the core idea behind CleanLink+ is solid. The reconciliation logic works, the API returns the right format, and the fuzzy matching is surprisingly effective.

The prototype also helped me catch design issues early ,like how picky OpenRefine is about JSON, or how tricky it is to tune match thresholds.

Most importantly, I now have a working base that I can build on. From here, I'll work on expanding the dataset, adding multi-field matching, writing more tests, and integrating directly with OpenRefine for real-world feedback.

For a first version, it does exactly what I need it to do: take messy city names and clean them up automatically. That's the goal ,and this prototype hits it.

References

1. **Cruz, J., Singh, N., and Janowicz, K.** (2022) *A review of address matching techniques for point of interest data*, Journal of Spatial Information Science, 24, pp. 1–25.

2. **Elmobark, A.** (2025) *Benchmarking Python string matching libraries for data cleaning*. [online] Available at: https://github.com/elmobark/string-matching-benchmark.

3. **GLAM Infrastructure Project** (2023) *Geocoding and Address Matching Techniques*. [online] Available at: https://glam-infra.org/geocoding-report.

4. **AddrMatcher** (2023) *RapidFuzz-based address matching with Australian GNAF*. [online] GitHub. Available at: https://github.com/datacube/addrmatcher.

5. **OpenRefine** (n.d.) *Reconciliation service API documentation*. [online] Available at: https://reconciliation-api.github.io/specs/latest.

6. **RapidFuzz** (n.d.) *RapidFuzz: Rapid string matching library*. [online] Available at: https://github.com/maxbachmann/RapidFuzz.

7. **DataMade** (2023) *dedupe: A Python library for accurate and scaleable entity resolution*. [online] GitHub. Available at: https://github.com/dedupeio/dedupe-examples.

8. **Wikidata** (n.d.) *Wikidata reconciliation service*. [online] Available at: https://wikidata.reconci.link/.